salesianos TRIANA | Colegio San Pedro

salesianos

TRIANA

# Proyecto Intermodular de Desarrollo de Aplicaciones Multiplataforma

Junio 2026

**Versión 1.0**

# HOJA DE CONTROL DE CAMBIOS

| Versión | Descripción | Fecha |
|---|---|---|
| 1.0 | Versión inicial del documento | 01/10/2025 |
| | | |
| | | |
| | | |

# Portada

# Índice

## 1. Introducción

Este es mi Proyecto Intermodular de Desarrollo de Aplicaciones Multiplataforma, realizado durante el curso 2025–2026. El trabajo consiste en el diseño y la implementación de un ecosistema completo de software formado por una API en Spring Boot, una aplicación web en Angular y una aplicación móvil en Flutter. El proyecto, denominado Precious Time, se centra en la gestión inteligente del tiempo personal y de las rutinas diarias, con el objetivo de ayudar al usuario a ser más consciente de en qué invierte su tiempo y cómo puede optimizarlo.

El sistema permite registrar actividades, agruparlas en rutinas, visualizar estadísticas y recibir sugerencias basadas en los hábitos del propio usuario. Para ello se ha diseñado una arquitectura por capas, desacoplando la lógica de negocio en una API REST y construyendo dos clientes independientes (web y móvil) que consumen los mismos servicios. Esta aproximación facilita la escalabilidad, el mantenimiento y la futura ampliación del proyecto, así como la posible publicación en entornos reales.

## 2. Plan de empresa

## 2.1. Idea de negocio

Precious Time se plantea como un servicio digital orientado a estudiantes, profesionales y personas que desean gestionar mejor su tiempo, priorizar tareas y reducir la procrastinación. La aplicación ofrece un panel unificado de rutinas y actividades, accesible tanto desde el móvil como desde el navegador, que permite planificar el día, registrar el tiempo real dedicado y analizar, mediante estadísticas, si se están cumpliendo los objetivos marcados.

La propuesta de valor se basa en tres pilares:

- Simplicidad de uso: interfaz limpia, centrada en rutinas y actividades, sin funciones innecesarias.

- Sincronización de datos: el usuario puede comenzar una actividad en el móvil y consultarla o modificarla desde la web gracias a la API centralizada.
- Concienciación y mejora continua: las estadísticas y resúmenes permiten reflexionar sobre el uso del tiempo y tomar decisiones para optimizarlo.

## 2.2. Público objetivo

El público objetivo principal son:

- Estudiantes de ciclos formativos y universidad que necesitan organizar estudios, proyectos y trabajos a la vez.
- Jóvenes profesionales que compaginan trabajo, formación continua y vida personal.
- Personas interesadas en productividad personal, autoorganización y hábitos.

Este segmento destaca por su alta adopción de tecnologías móviles, uso intensivo de aplicaciones y necesidad real de herramientas que les ayuden a estructurar su día a día.

## 2.3. Modelo de negocio (visión inicial)

En una fase inicial, Precious Time se plantea como una aplicación gratuita para captar usuarios, validar funcionalidades y recopilar feedback. A medio plazo, podría evolucionar hacia un modelo freemium, manteniendo las funciones básicas gratuitas e introduciendo características avanzadas como:

- Estadísticas avanzadas y comparativas históricas.
- Recomendaciones personalizadas basadas en patrones de uso.
- Exportación de datos e integración con otras herramientas (calendarios, gestores de tareas, etc.).

El coste principal se centra en el desarrollo y mantenimiento de la infraestructura backend (API y base de datos) y en la evolución continua de las aplicaciones web y móvil.

# 3. Descripción del sistema

## 3.1. Visión general

Precious Time está formado por tres componentes principales:

- Una API REST desarrollada en Spring Boot, que centraliza la lógica de negocio, la gestión de usuarios, rutinas y actividades, y el acceso a datos.

- Una aplicación web desarrollada en Angular, que actúa como cliente de escritorio/navegador y ofrece una interfaz completa para gestión y consulta.
- Una aplicación móvil desarrollada en Flutter, enfocada en el uso diario y rápido desde smartphone, con acceso a las mismas funciones clave.

Los tres componentes se comunican mediante peticiones HTTP a la API, compartiendo el mismo modelo de datos y las mismas validaciones de negocio.

## 3.2. Objetivos funcionales

A nivel funcional, el sistema permite al usuario:

- Registrarse, autenticarse y gestionar su perfil personal.
- Crear, modificar y eliminar rutinas (por ejemplo: estudio, deporte, descanso, trabajo).
- Definir actividades dentro de cada rutina, con duración prevista, horarios y etiquetas.
- Registrar el tiempo realmente dedicado a las actividades.
- Consultar estadísticas de uso del tiempo por día, semana o tipo de rutina.

En futuras iteraciones, el sistema podría incluir recomendaciones automáticas sobre cómo reorganizar las rutinas para conseguir un mejor equilibrio entre estudio, trabajo y ocio.

## 3.3. Arquitectura general

La arquitectura sigue un enfoque cliente-servidor:

- La API en Spring Boot expone endpoints REST para gestionar usuarios, rutinas, actividades y estadísticas, apoyándose en un modelo de entidades y repositorios conectados a una base de datos relacional.
- La aplicación web en Angular consume estos endpoints, implementando servicios HTTP, componentes y vistas que organizan la interfaz según módulos funcionales (autenticación, rutinas, estadísticas, configuración).
- La aplicación móvil en Flutter consume igualmente la API mediante servicios de red, gestionando estados y pantallas específicas para registro de tiempo, listado de rutinas y visualización de datos.

Esta separación permite desarrollar y desplegar de forma independiente el backend y los clientes, reutilizando la lógica de negocio central en la API y manteniendo coherencia en los datos mostrados en todos los dispositivos.

# 4. Modelado y diseño

## 4.1. Análisis y casos de uso

En la fase de análisis se identificaron los actores principales del sistema y los casos de uso asociados. El actor principal es el Usuario registrado, que interactúa tanto desde la aplicación web como desde la móvil para gestionar su tiempo. A partir de este actor se definieron los siguientes casos de uso principales:

- Registrarse e iniciar sesión en la aplicación.
- Gestionar su perfil de usuario (datos básicos, preferencias de idioma o tema, por ejemplo).
- Crear, modificar y eliminar rutinas.
- Crear, modificar y eliminar actividades asociadas a una rutina.
- Registrar el tiempo invertido en una actividad concreta.
- Consultar estadísticas y resúmenes de uso del tiempo.

Estos casos de uso se representan en un diagrama donde el actor Usuario se relaciona con las funcionalidades principales del sistema (autenticación, gestión de rutinas, gestión de actividades y estadísticas), sirviendo como base para el diseño posterior de la API y de las interfaces de usuario.

## 4.2. Modelo de dominio y entidades

Del análisis anterior se obtuvo un modelo de dominio que identifica las entidades principales y sus relaciones. Las entidades más importantes son:

- Usuario: representa a la persona que utiliza la aplicación, con atributos como identificador, nombre, correo, contraseña cifrada y posibles preferencias.
- Rutina: agrupa un conjunto de actividades relacionadas con un ámbito concreto (estudio, trabajo, deporte, etc.) y se asocia a un Usuario.
- Actividad: acción concreta dentro de una rutina, con nombre, descripción, duración prevista y estado (pendiente, en curso, completada).

- Registro de tiempo o Estadística: almacena la información sobre el tiempo real invertido en cada actividad, permitiendo posteriormente generar informes y gráficos.

Las relaciones fundamentales son: un Usuario puede tener varias Rutinas, cada Rutina puede contener varias Actividades y cada Actividad puede generar múltiples registros de tiempo a lo largo del uso de la aplicación.

## 4.3. Diseño de la interfaz (Figma / prototipos)

Para la parte de diseño de interfaz se elaboraron prototipos de baja y media fidelidad, tanto de la aplicación web como de la móvil. En ellos se definieron la distribución de componentes, la navegación principal y la jerarquía visual de la información. En la versión web se prioriza un panel central con el listado de rutinas y, al seleccionar una, se muestran sus actividades y estadísticas relacionadas.

En la aplicación móvil se optó por una navegación mediante barras y pestañas, facilitando el acceso rápido a:

- Pantalla de inicio con resumen del día.
- Listado de rutinas y actividades.
- Estadísticas básicas.

Estos prototipos sirvieron como guía para la implementación en Angular y Flutter, asegurando coherencia visual y de experiencia de usuario entre ambas plataformas.

---

# 5. Implementación general

## 5.1. Tecnologías utilizadas

El proyecto se ha implementado utilizando las siguientes tecnologías principales:

- Backend: Spring Boot para la creación de la API REST, con Java como lenguaje y uso de anotaciones para definir controladores, servicios y repositorios.
- Web: Angular como framework de desarrollo frontend, con TypeScript, HTML y CSS para la construcción de componentes y vistas.
- Móvil: Flutter como framework multiplataforma y Dart como lenguaje para el desarrollo de la aplicación móvil.
- Base de datos: sistema relacional (por ejemplo, MySQL o similar) accesible desde la API mediante repositorios JPA.

Esta combinación de tecnologías permite separar claramente la lógica de negocio del backend de la presentación en los clientes web y móvil, y facilita la escalabilidad y el mantenimiento del sistema.

## 5.2. Estructura general de los proyectos

Cada uno de los tres proyectos (API, web y móvil) se ha organizado por paquetes o módulos para mejorar la legibilidad y modularidad del código. En la API se han separado controladores, servicios, repositorios y entidades, siguiendo una arquitectura en capas clásica en aplicaciones Spring Boot. En la web Angular se han creado módulos y componentes específicos para autenticación, rutinas y estadísticas. En la app móvil Flutter se han definido pantallas, modelos y servicios de acceso a la API, organizando el código en carpetas según responsabilidad.

---

# 6. Descripción de paquetes y clases

## 6.1. API – Spring Boot

En el proyecto de la API se distingue una estructura típica en capas:

- Paquete de entidades (por ejemplo, `model` o `entity`):
  - Clase `User`: representa al usuario del sistema, con campos para id, nombre, correo y contraseña cifrada.
  - Clase `Routine`: modela una rutina de usuario, con relación a `User` y campos como nombre y descripción.
  - Clase `Activity`: representa una actividad concreta dentro de una rutina, con atributos como nombre, duración prevista y estado.
  - Clase `TimeRecord` o `Statistic`: almacena los datos de tiempo real invertido en cada actividad, con fecha, duración y referencias a la actividad correspondiente.
- Paquete de repositorios (por ejemplo, `repository`):
  - Interfaces que extienden de `JpaRepository` u otra interfaz de Spring Data para cada entidad (`UserRepository`, `RoutineRepository`, `ActivityRepository`, etc.), proporcionando métodos de acceso a la base de datos.
- Paquete de servicios (por ejemplo, `service`):
  - Clases de servicio como `UserService`, `RoutineService` o `ActivityService` encapsulan la lógica de negocio y coordinan el acceso a los repositorios.

    - Aquí se validan datos, se aplican reglas de negocio (por ejemplo, evitar duplicados, comprobar pertenencia de rutinas a usuarios, etc.).
- Paquete de controladores (por ejemplo, `controller`):
    - Clases como `UserController`, `RoutineController` o `ActivityController` exponen endpoints REST para que los clientes web y móvil puedan realizar operaciones CRUD sobre usuarios, rutinas y actividades.
    - Estos controladores reciben las peticiones HTTP, delegan en los servicios y devuelven respuestas JSON.

Esta organización facilita el mantenimiento y la extensión de la API, permitiendo añadir nuevas funcionalidades manteniendo la separación de responsabilidades.

## 6.2. Aplicación web – Angular

En el proyecto Precious-Time-Web se utilizan módulos y componentes de Angular para estructurar la interfaz y la lógica de presentación. Los elementos más destacados son:

- Módulos de la aplicación:
    - Un módulo principal que agrupa la configuración base de la aplicación y el enrutado.
    - Módulos específicos para funcionalidades como autenticación y gestión de rutinas, que cargan componentes y servicios relacionados.
- Componentes:
    - Componente de login/registro: gestiona los formularios de autenticación del usuario y se comunica con la API para validar credenciales.
    - Componente de listado de rutinas: muestra las rutinas del usuario en forma de tabla o tarjetas, permitiendo crear, editar y eliminar rutinas.
    - Componente de detalle de rutina: muestra las actividades asociadas a una rutina seleccionada y permite gestionar dichas actividades.
    - Componentes de estadísticas: representan gráficamente o en forma de listas los datos de tiempo registrado.
- Servicios:
    - Servicios Angular encargados de realizar las peticiones HTTP a la API (por ejemplo, servicios de usuario, rutinas y actividades), centralizando la lógica de acceso a datos.

La aplicación web actúa así como un cliente rico que consume la API, presentando la información de forma clara y permitiendo al usuario gestionar cómodamente sus rutinas y actividades desde el navegador.

## 6.3. Aplicación móvil – Flutter

En el proyecto Precious-Time-Mobile se organiza el código en carpetas para separar modelos de datos, servicios y pantallas. Los elementos más representativos son:

- Modelos (models):
  - Clases Dart que representan entidades como Usuario, Rutina y Actividad, con sus campos y métodos de serialización/deserialización JSON para comunicarse con la API.
- Servicios (services):
  - Clases responsables de realizar peticiones HTTP a la API (por ejemplo, un servicio de autenticación y otro de gestión de rutinas y actividades).
  - Estos servicios encapsulan la lógica de red y devuelven datos ya mapeados a los modelos.
- Pantallas (screens/pages):
  - Pantalla de inicio de sesión y registro.
  - Pantalla de listado de rutinas del usuario, desde la que se accede al detalle de cada rutina.
  - Pantalla de detalle de rutina con sus actividades, donde se puede registrar el tiempo real invertido.
  - Pantalla de estadísticas básicas, con información de uso del tiempo.

Gracias a esta estructura, la aplicación móvil puede ofrecer una experiencia de uso rápida y centrada en el día a día, consumiendo los mismos servicios de la API que la aplicación web y manteniendo los datos siempre sincronizados.